

Adatbázis-tervezés: A relációs adatmodell fogalma. Az egyed-kapcsolat diagram és leképezése relációs modellre, kulcsok fajtái. Funkcionális függőség, a normalizálás célja, normálformák

ALAPOK

- **Relációs adatmodell:** táblákban tárolja az adatokat és az ezek a táblák között kapcsolatok vannak
- **Egyed-kapcsolat diagram:** segítenek az adatbázis struktúrájának és az entitások közötti kapcsolatok áttekintésében
- **Funkcionális függőség és normalizálás:** csökkenti a redundanciát és optimalizálja az adatbázis struktúráját

ADATBÁZIS-TERVEZÉS: A RELÁCIÓS ADATMODELL FOGALMA

Relációs adatmodell

- adatbázis-kezelési megközelítés, mely az adatokat táblázatok formájába tervezi, melyek közötti kapcsolatokat matematikai relációk segítségével lehet definiálni és kezelni:
 - **Adatok táblázatokban**
 - relációs adatmodell az adatokat táblázatokba szervezi
 - egy táblázat egy adott típusú adatot reprezentál, pl: “Vevők” tábla tartalmazza a vevőkhöz tartozó információkat (név, cím)
 - minden táblázat sorokból (**rekordokból**) és oszlopokból (**attribútumokból**) épül fel
 - **Kapcsolatok**
 - a táblázatok között vannak / lehetnek kapcsolatok, melyek matematikai relációk segítségével definiálhatók
 - pl: “Vevők” táblázathoz tartozhat egy “Rendelések” táblázat a vevő azonosítója alapján
 - segítenek az adatok összekapcsolásában és az adatok közötti összefüggések kezelésében
 - **Kulcsok**
 - egyes rekordok, sorok azonosításához használjuk a kulcsokat
 - ezen felül segítenek a kapcsolatok meghatározásában, azaz a táblák összekapcsolásában
- példa:
 - tegyük fel, hogy van két táblánk egy adatbázisban:
 - ****Vevők**** tábla
 - vevőazonosító: {1, 2, 3}
 - név: {Alice, Bob, Carol}
 - ****Rendelések**** tábla
 - rendelésazonosító: {101, 102, 103}
 - vevőazonosító: {1, 2, 1}
 - terméknév: {Toll, Füzet, Radír, Ceruza}

- a két tábla közötti reláció az, hogy melyik vevő melyik rendelést adta le
 - matematikai relációként: $\text{Rendelések} = \{(1, 101), (2, 102), (1, 103)\}$
 - ez azt jelenti, hogy a vevőazonosító (1) (Alice) két rendelést adott le ((101) és (103)) és a vevőazonosító (2) (Bob) egy rendelést adott le ((102))
- miért hasznos a relációs adatmodell?
 - **Adatok egysége és integritása**
 - a relációs adatmodell gondoskodik az adatok típusának konzisztenciájáról és integritásáról
 - ha pl. a vevő nevének típusa szöveg (VARCHAR) akkor az adatbázisban nem engedélyezett a táblának olyan rekordokat tartalmaznia ahol a vevő neve pl: szám, vagy dátum
 - **Redundancia csökkentése**
 - a táblázatok és kapcsolatok használatával minimalizálható az adatok felesleges ismétlése
 - kulcsokkal biztosítom, hogy ne legyen ugyanaz az adat 2x benne, inkább referálok másik táblára
 - **Rugalmasság és Bővíthetőség**
 - az adatbázis szerkezetének könnyű módosítása új táblák és attribútumok hozzáadásával, anélkül, hogy az egész adatbázist át kellene tervezni
 - **Adatbiztonság**
 - a hozzáférés pontos szabályozása biztosítja, hogy csak az engedélyezett felhasználók érjenek el bizonyos adatokat és végezzenek bizonyos műveleteket
 - hozzáférési szabályok és engedélyezés
- **Elnevezések**
 - **Adatbázis (db)**
 - adatok struktúrált gyűjteménye, melyet bizonyos formátumban tárolnak és kezelnek
 - **Rekord (feljegyzés)**
 - az adatbázis alapvető adategysége, általában struktúra felépítésű
 - **Értéktartomány (domain)**
 - egy attribútum lehetséges értékeinek halmaza az adatbázisban
 - adottak $D_1, D_2, \dots, D_n$ tartományok, akkor az R reláció:

$$R \subseteq D_1 \times D_2 \times \dots \times D_n$$
 - jelölése x esetéend $\text{dom}(x)$, pl: VARCHAR(255)

Adatbázis alkalmazások felépítése

- felhasználói felület
 - az a réteg mellyel a végfelhasználók közvetlen interakcióba lépnek
 - pl: grafikus felület, webes interfészek
- célalkalmazásként készült program
 - a felhasználói felület mögöt futó alkalmazások, melyek adatokat dolgoznak fel és DBMS-el kommunikálnak
- logikai adatstruktúra (adatmodell)
 - az adatbázis logikai szerkezete, mely meghatározza a táblák, mezők és kapcsolatok struktúráját
- DBMS (DataBase Management System - adatbázis-kezelő rendszer)
 - szoftver, mely az adatbázis kezelését, tárolását és lekérdezését végzi
 - pl: MySQL, PostgreSQL
- fizikai adattárolás
 - adatok tényleges tárolása a háttértárolókon

Adatok típusai

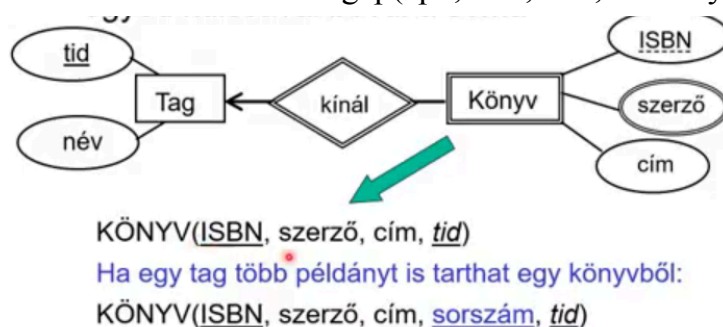
- **Egyszerű (atomi) adat:** szám, string, dátum, logikai érték
- **Összetett adat**
 - egyszerű adatokból képezhető
 - halmaz
 - egynemű (azonos típusú) elemek összesége, amelyek nem rendelkeznek sorrenddel vagy hierarchiával
 - pl: könyvek genre-i (típusai, stílusai)
 - lista
 - egynemű elemek rendezett sorozata
 - pl: könyvek szerzői
 - struktúra
 - különböző típusú elemek rendezett sorozata, amelyek összetartoznak
 - pl: (lakcím = (helység, utca, házsám))
 - + a fentiek kombinációi
- **NULL:** definiálatlan adat (NEM azonos a nulla értékkel)

EGYED-KAPCSOLAT DIAGRAM ÉS LEKÉPZÉSE RELÁCIÓS MODELLRE

- az Egyed-Kapcsolat diagram (ER) 3 alapvető eleme az **egyed**, az **attribútum** és a **kapcsolat**

Egyed

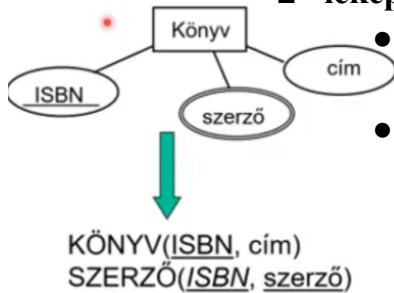
- valós világbeli dolog amit tulajdonságokkal (attribútumok) akarunk leírni (pl: könyv, olvasó)
- jelölése: téglalap
- **leképzése relációs modellre**
 - minden egyedhez felírunk egy relációs sémát (tábla) és az összetett attribútumokat szétbontjuk
- egyed (entitás) fajtái
 - **erős egyed**
 - kulcs attribútuma egyértelműen azonosítja, azaz meghatározza
 - **leképzése relációs modellre**
 - a séma neve egyenlő az egyed nevével
 - a séma attribútumai megegyeznek az egyed attribútumával (kulcs is)
 - pl: Személy(személyszám, név, születési dátum)
 - **gyenge egyed**
 - tulajdonságai, attribútumai nem határozzák meg egyértelműen
 - ilyenkor szaggatott vonalal aláhúzzuk a kulcsokat
 - az egyedet a meghatározó kapcsolatai alapján tudjuk azonosítani, jelölés dupla vonalas téglalap
 - **leképzése relációs modellre**
 - a séma neve egyenlő az egyed nevével itt is
 - a séma attribútumai megegyeznek az egyed attribútumával, ám ki kell bővíteni egy olyan attribútummal mely biztosítja az egyediséget, ez általában egy meghatározó kapcsolatban álló másik egyed kulcsa
 - 2 kulcs is lehet egyszerre, az új kulcs és az eredeti gyenge kulcs
 - pl: ha egy tulajdonosnak több, azonos gépe lehet, akkor ezeket egy sorszám attribútummal különböztetjük meg:
Tulajdonos(személyszám, név, lakcím)
Számítógép(cpu, ram, hdd, személyszám, sorszám)



Attribútum

- az egyedről tárolandó adat(ok), azaz az egyed jellemzője, tulajdonsága
 - pl: könyv egyed esetén a címe és a szerzője
- jelölése: elipszis
- **relációs adatmodell kulcsai**
 - **szuperkulcs**
 - egy olyan attribútumhalmaz, ami minden sorban egyedi és az összes sort egyértelműen azonosítja
 - ez még tartalmazhat felesleges elemeket (pl: id, név is szuperkulcs pedig név felesleges benne)
 - **kulcs (minimális szuperkulcs)**
 - az a minimális attribútum halmaz (akár egy attribútum) mely egyértelműen meghatároz egy egyedet
 - extra jelölés, hogy az attribútum neve alatt aláhúzás van
 - ha egy attribútumból áll akkor egyszerű, ha több attribútum alkotja akkor összetett kulcsról beszélünk
 - egy táblának több kulcsjelöltje is lehet
 - **elsődleges kulcs**
 - a relációs sémában meghatározott kulcsok közül az egyik az elsődleges lesz (mindig csak 1 darab ilyen van)
 - a szuperkulcsok közül az a kulcs az elsődleges, mely az adott táblán belül egyértelműen azonosítja a sorokat és garantálja, hogy mindegyik sor rendelkezik egy olyan egyedi azonosítóval, melyet a rendszer használhat
 - **külső kulcs**
 - egy relációsséma attribútumainak részhalmaza akkor idegen kulcs, ha az egy másik séma kulcsára hivatkozik
 - jelölése dőlt betű, vagy nyíllal jelöljük, hogy mire mutat
 - csak NULL vagy olyan érték lehet mely a hivatkozott táblában is szerepel (hivatkozási integritás)
- speciális attribútumok
 - **összetett attribútum**
 - az attribútumnak vannak attribútumai
 - pl: lakím esetén a lakcímnek lehet az attribútuma a város, cím, házszám
 - **leképzése relációs adatmodellre**
 - részattribútumaival, elemeivel helyettesítjük a relációs sémába az adott attribútumot
 - pl: lakcímhez tartozik, az város, cím, házszám, akkor a sémába ez a három kerül be a lakcím helyett, Személy(személyi szám, név, születési dátum, utca, házszám, város)

- többértékű attribútum
 - egy attribútum rendelkezhet több értékkel
 - pl: könyvnek lehet több szerzője
- jelölése a dupla vonalas elipszisz

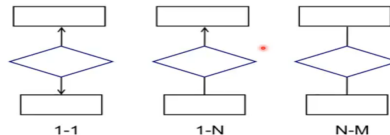


■ leképzése relációs adatmodellre

- 1. mód: egy hosszú stringként tároljuk a keresés lassú így, ezért nem használjuk
- 2. mód: egy új táblát, relációs sémát veszünk fel (indexeljük) csak annak a tagnak
 - egyed kulcsa (külső kulcsként) és maga az adott tag

Kapcsolat

- egyedek közti összefüggés (pl: könyv és olvasó között a kölcsönzés a kapcsolat)
- indokolt esetben egy kapcsolathoz rendelhetünk attribútum(ok)-t
- jelölése: rombusz
- kapcsolatok típusai
 - N-M (N-N)



- minden A egyedhez több (N darab) B egyed és minden B egyedhez több (M darab) A egyed is tartozhat
- jelölése: nincs semmilyen nyíla a kapcsolatnak
- pl: Hallgató és Kurzus (egy hallgató több kurzust vehet föl és egy kurzust is több hallgató vehet fel)
- **leképzése relációs modellre**
 - felvesszük az egyedek relációs sémáját amiket a kapcsolat kapcsol össze, majd a kapcsolat sémáját felvesszük melynek attribútumai a kapcsolat attribútumai és a résztvevő egyedek kulcs

- extra kapcsoló tábla



- 1-1

- minden A egyedhez csak egy B egyed és minden B egyedhez csak egy A egyed tartozhat
- jelölés: nyíl mindkét egyed felé
- pl: Osztályfőnök és Osztály

■ leképzése relációs modellre

- hasonlóan is eljárhatunk mint az N-M kapcsolat esetében
- de megoldható annyival, hogy bármelyik egyedhez felvesszük a kapcsolat attribútumát és a másik egyed kulcsát, mint idegen kulcs
 - beolvasszuk a kapcsolat sémáját bármelyik egyedbe



○ 1-N

- minden A egyedhez csak egy B egyed tartozhat, de minden B egyedhez akár több A egyed is tartozhat
- jelölés: nyíl az A (az 1 felé) mutat
- pl: Tanszék és Oktató (egy tanszéken több oktató van) vagy Dolgozó és Számítógép (1 géphez több dolgozó is tartozhat, de egy dolgozónak 1 gépe van)
- leképzése relációs modellre

- szintén hasonlóan eljárhatunk mint az N-M kapcsolat esetén
- de megoldható annyival, hogy az N (több) oldalú egyedhez felvesszük a kapcsolat attribútumait és az 1 oldalú egyed kulcsát, mint idegen kulcs
 - tehát itt nem veszünk így fel egy extra táblát, csak az N oldalú egyedet bővítjük ki
 - ezt **konszolidációnak** hívjuk ami annyit jelenet, hogy azonos kulcs esetén 2 séma összevonható



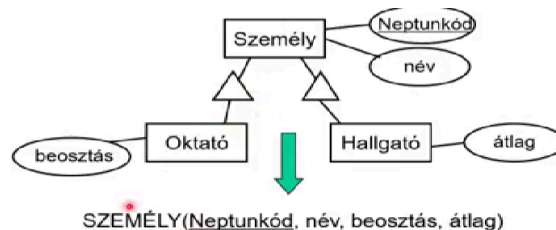
- meghatározó kapcsolat
 - ha az egyed gyenge a meghatározó kapcsolat alapján tudjuk azonosítani
 - ilyenkor N-N típusú kapcsolat nem lehetséges, mert akkor nem tudnánk meghatározni
 - jelölése: dupla vonalas rombusz
- specializáló kapcsolat
 - egy általános egyed altípusainak modellezése (pl: neptunban egy felhasználónak lehet oktatói és hallgatói altípusa)
 - jelölése: az összeköttetés vonalára egy háromszöget kell rajzolni, melynek csúcsa amivel érintkezik a fő egyed fele mutat (pl: neptun felhasználó felé)

- **leképzése relációs modellre**

- 3 lehetőség van a megvalósítására

- 1. közös tábla felvétele a két alegyed attribútumainak uniójával**

- itt biztos lesz NULL érték a táblában (ha nem tartozik mindkét alosztályba bele)
 - furcsa mivel NULL azt is jelenti, hogy nincs még kitöltve
 - táppazarló is, feleslegesen tároljuk a NULL értékeket
- akár egy típus is hozzáadható (pl: hallgató akkor 1, ha oktató akkor 2)



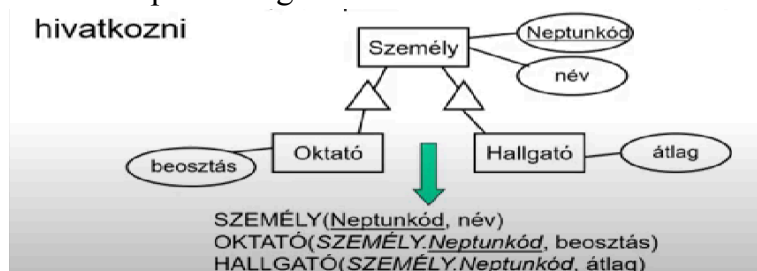
- 2. altípusonként új sémákat hozunk létre, az őstípus attribútumaival és az altípus saját attribútumaival**

- az új sémák része lesz a főtípus összes attribútuma és az altípus attribútumai, **kulcsa a főtípus kulcsa lesz**
- hibája, hogy kereséskor az összes táblát át kell fésülni
 - pl: ha nem tudjuk a személy hallgató vagy oktató, csak a neptun kódját akkor mindkét táblát meg kell nézni



- 3. altípusonként új sémákat hozunk létre a saját attribútumaival, illetve az őstípus kulcsa külső kulcs lesz**

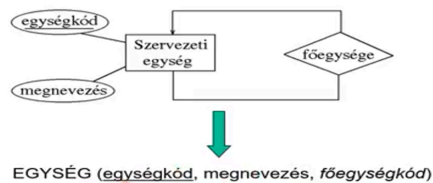
- új sémákat hozunk létre, a séma része viszont csak az adott altípus attribútuma, a főtípus kulcsa pedig külső kulcsként fog szerepelni
- itt viszont fenn áll az, hogyha a nevét is lekérnénk
 - pl az átlag mellé akkor azt 2 külön táblából érem el



- sokágú kapcsolat

- több egyedet kapcsol össze, de szétbontható
- pl. a vásárlás kapcsolat összekapcsolhatja a raktár, vevő és árú egyedeket

- egy egyed teljesen részt vesz a kapcsolatban
 - pl csak akkor létezik egy Hirfolyam egyed, ha Kedvelés kapcsolatban vele össze van legalább egy Felhasználó egyed kapcsolava, ami létezik is
 - jelölése dupla vonallal kötjük a kapcsolathoz azt az egyedet mely csak akkor létezik, ha a másik egyedből legalább 1 van ami kapcsolatban áll vele
- önmagával kapcsolatban lévő egyedek
 - pl egy dolgozónak van egy főnöke aki egy másik dolgozó egyed (1-N), saját magának főnöke nem lehet
 - pl egy csapat meccset játszik egy másik csapat egyeddel (N-M)
 - **leképzésre relációs modellel**
 - hasonló módon megoldható, felvesszük az egyed kulcsát más néven(névütközés elkerülése), mint idegen kulcs (pl csapat kód esetén ellenél csapat kódként)



FUNKCIONÁLIS FÜGGŐSÉG ÉS NORMÁL FORMÁK

Funkcionális függőség

- azt jelenti, hogy egy attribútumhalmaz értéke meghatározza egy másik attribútumhalmaz értékét egy relációs sémában
 - ha $R(A_1, A_2, A_3, \dots, A_n)$ egy séma és P, Q részhalmaza az R attribútumainak $\{A_1, A_2, \dots, A_n\}$, akkor P -től funkcionálisan függ a Q , ha bármely R feletti T tábla esetén valahányszor megegyezik két sor P -n, akkor Q -n is megegyezik
 - jelölése $P \rightarrow Q$ (P -től funkcionálisan függ a Q)
 - példa
 - $\{\text{osztálykód}\} \rightarrow \{\text{osztálynév, vezAdószám}\}$
 $\{\text{adószám}\} \rightarrow \{\text{név, lakcím, osztálykód, osztálynév, vezAdószám}\}$
 - az adószám funkcionálisan függ az osztálykódtól, mivel ha két rekordban az osztálykód és a vezető adószám értéke megegyezik, akkor az adószám értéke is megegyezik
 - ezért mondhatjuk, hogy az osztálykód meghatározza az adószámot
- **Teljes függőség (2NF esetén)**
 - $R(A)$ esetén X, Y részhalmaza A -nak és $X \rightarrow Y$, azaz Y funkcionálisan függ X -től
 - ilyen esetben akkor van szó teljes függőségről ha X egyetlen valódi részhalmazától sem függ Y , azaz ha X -ből elhagyunk egy attribútumot akkor a függőség már nem áll fenn
 - ha K egy kulcs akkor A teljesen függ a K -tól
- **Tranzitív függés (3NF esetén)**
 - $R(A)$ esetén, ha X, Z részhalmaza az A -nak és $X \rightarrow Z$, azaz X -től függ a Z
 - X -től akkor függ tranzitívan a Z , ha van olyan Y ami részhalmaza az A -nak és érvényes az, hogy $X \rightarrow Y$ és $Y \rightarrow Z$, de $Y \not\rightarrow X$ és $Y \rightarrow Z$ függés teljesen nemtriviális, azaz Y és Z metszete a semmi
 - ha ezek nem állnak fenn, akkor Z közvetlenül függ X -től
 - tranzitív függés = sorba egymástól függenek pl: SzemélyiSzám $\rightarrow$ ProjektId és ProjektId $\rightarrow$ Projektnev
- **triviális és nemtriviális funkcionális függés**
 - ha Q részhalmaza P -nek, $P \rightarrow Q$ esetén triviális, különben nemtriviális a funkcionális függés
- **teljesen nemtriviális funkcionális függés**
 - ha $Q \cap P = \emptyset$, azaz üreshalmaz

Amstrong axiómák

- Reflexivitás: Y részhalmaza X -nek akkor $X \rightarrow Y$
- Bővítés: $X \rightarrow Y$, akkor $X \cup Z \rightarrow Y \cup Z$
- Transzitivitás: $X \rightarrow Y$ és $Y \rightarrow Z$, akkor $X \rightarrow Z$

Normálformák

- ha helyesen írjuk fel az egyed-kapcsolat modellt, akkor általában optimális (redundanciamentes) relációs adatbázist kapunk
 - de mivel nincs erre garancia ezért formálisan vizsgálni kell a relációsémákat, oly módon amely detektálni tudja a redundanciát és optimalizálást, normalizálást tesz lehetővé
- redundancia aktualizálási anomáliákat okozhat

- **módosítási anomália**

- ha egy adat többször szerepel, és változik, mindenhol át kell írni, különben inkonzisztens lesz a rendszer
- pl az ábra alapján: osztály neve vagy vezetője megváltozik

- **beszúrási anomália**

- új rekord felvétele esetén, mondjuk elírunk valamit
- pl az ábra alapján: osztály nevet elírjuk (tervezési helyett mondjuk tervezőt adunk meg)

- **törlési anomália**

- egy adat törlésével más, hasznos információ is elveszik
- pl: ha egy Osztály összes dolgozója törlődik, akkor az Osztály is

elveszik	Név	Adószám	Cím	Osztálykód	Osztálynév	VezAdószám
	Kovács	1111	Pécs, Vár u.5.	2	Tervezési	8888
	Tóth	2222	Tata, Tó u.2.	1	Munkaügyi	3333
	Kovács	3333	Vác, Róka u.1.	1	Munkaügyi	3333
	Török	8888	Pécs, Sas u.8.	2	Tervezési	8888

- **elsődleges és másodlagos attribútum**

- elsődleges attribútumról akkor van szó ha az adott attribútum része a séma egyik kulcsának
- másodlagos attribútumról akkor van szó ha az adott attribútum nem része a séma egyik kulcsának se

- **1NF**

- egy relációsséma 1NF-be van, ha az attribútumok értéktartománya csak egyszerű (atomi) adatokból áll
 - atomi adat = nem tartalmaz listát vagy struktúrát
- elméletileg minden relációs modellre teljesül az 1NF, mert leképezés során megtörténik az összetett attribútumok szétbontása

- **2NF**

- egy relációsséma 2NF-ben van, ha **a másodlagos attribútumok teljesen függnének a séma bármely kulcsától** (nincs benne részleges függőség)
 - tehát nem függhet egy másodlagos attribútum a kulcsnak csak egy részétől (ilyenkor a függőségekből elhagyhatjuk a más kulcs attribútumokat és csak a másodlagosakat kell nézni)
- ha a sémának minden kulcsa egyelemű (1 attribútumból áll) vagy nincs másodlagos attribútuma akkor már automatikusan 2NF-be van

- 2NF-re hozás
 - a normál formát sértő függés szerint felbontjuk a sémát (Heath tétele szerint)
 - új séma felvétele a függésben szereplő attribútumokkal
 - az eredeti sémából töröljük a függés jobb oldalán szereplő attribútumokat, a függés bal oldalán lévő attribútumokat az eredeti sémában külső kulcsnak is beállítjuk
- példa a fentebbi ábrához
 - táblák:
 - DOLGPROJ(Adószám, Név, Projektkód, Óra, Projektnév, Projekthely)
 - függőségei:
 - Adószám → Név
 - Projektkód → {Projektnév, Projekthely}
 - {Adószám, Projektkód} → Óra
 - kulcsok: {Adószám, Projektkód}
- séma nincs 2NF-ben, mert vannak normál formát sértő függések:
 - Adószám → Név
 - Név csak Adószám-tól függ, vagyis nem függ a teljes kulcstól
 - Projektkód → {Projektnév, Projekthely}
 - a Projektnév és a Projekthely pedig csak a Projektkód-tól függ
- dekompozícióval szétválasztjuk a 2-es normált sértő függéseket:
 - Adószám → Név esetén létre jön egy Dolgozó tábla és az eredeti táblából eltűnik a Név, az Adószám meg egy külső kulcs is lesz
 - Projektkód → {Projektnév, Projekthely} esetén létre jön egy Projekt tábla és eltűnik az eredeti táblából a Projektnév és Projekthely, míg a Projektkód egy külső kulcs lesz
- végeredmény:

DOLGPROJ(Adószám, Projektkód, Óra)

DOLGOZÓ(Adószám, Név)

PROJEKT(Projektkód, Projektnév, Projekthely)
- 3NF
 - egy relációsséma 3NF-ben van, ha **a másodlagos attribútumok közvetlenül függnak a séma bármely kulcsától** (nincs tranzitív függés)
 - ha nincs másodlagos attribútum akkor az a séma automatikusan 3NF-ben van
 - ha 3NF-be van akkor tudjuk, hogy 2NF-be is van
 - 3NF-re hozás (ugyanaz mint 2NF esetén)
 - a normál formát sértő függés szerint felbontjuk a sémát
 - új séma felvétele a függésben szereplő attribútumokkal
 - az eredeti sémából töröljük a függés jobb oldalán szereplő attribútumokat, a függés bal oldalán lévő attribútumokat az eredeti sémában külső kulcsnak is beállítjuk

- példa

- táblák:

DOLGPROJ(Adószám, Tajsám, Név, Projektkód, Projektnév)

- kulcsok: { Adószám } és { Taj szám }

- *kikötés*: egy dolgozó csak 1 projekten dolgozhat, ha többen dolgoznak egy projekten akkor a projekt neve ismétlődik

- függőségek:

{Adószám} → { TAJ szám, Név, Projektkód }

{Projektkód} → {Projektnév}

- séma 2NF-be van mivel egyszerű kulcsok vannak

- viszont nincs 3NF-be, mert van tranzitív függés:

{Adószám} → {Projektkód} → {Projektnév}

- javításra Heath-tétele alapján felbontjuk és a végeredmény:

DOLGOZÓ(Adószám, Tajsám, Név, Projektkód)

PROJEKT(Projektkód, Projektnév)

- **Boyce-Codd normálforma (BCNF)**

- ez a 3NF szigorítása, egy reláció BCNF-ben van, ha minden $X \rightarrow Y$ függőség esetén X superkulcs

- ez kezeli az átfedő összetett kulcsok ritka problémáit

DOLGOZÓ(adószám, TAJ szám, projektkód)

Egy dolgozó több projekten dolgozhat.

Ha valaki több projekten dolgozik, akkor a TAJ szám ismétlődik.

Kulcsok: {adószám, projektkód}, {TAJ szám, projektkód}

A séma 3NF-ben van (nincs másodlagos attribútum).

BCNF-et sértő függés: {adószám} → {TAJ szám} (adószám nem superkulcs)

Felbontás Heath-tétele alapján:

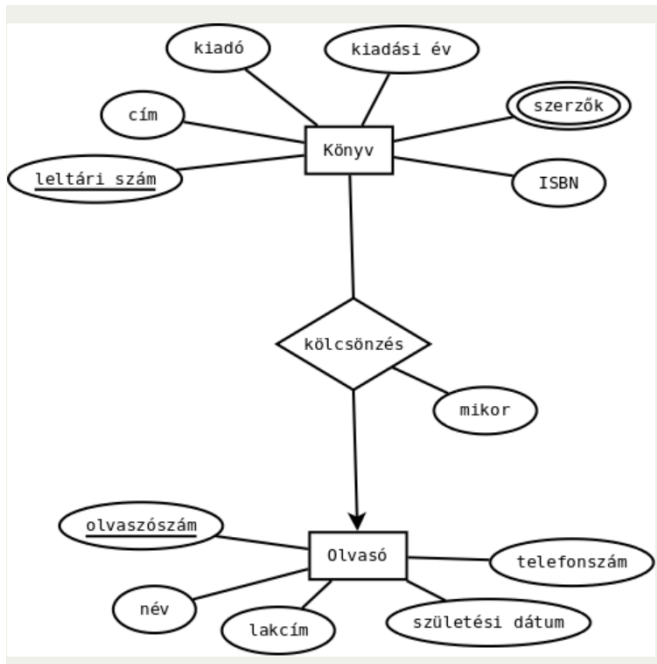
C = {adószám}

D = {TAJ szám}

B = {projektkód}

DOLGOZÓ(adószám, TAJ szám)

PROJEKT(adószám, projektkód)



Tervezzünk egy könyvtári adatbázist! A könyvtárban a könyveket leltári számmal látják el, amely egyértelműen azonosítja a könyv példányt. Egy adott könyvből több példány is rendelkezésre áll a könyvtárban. A leltári számon túl a könyvről eltárolják a könyv címét, szerzőt (mivel lehetséges, hogy több szerzője van a könyvnek), az ISBN számát, a kiadót és a kiadási évet. A könyvtári adatbázisban eltárolják az olvasókat is. Minden olvasó egyedi olvasószámmal rendelkezik, ezen kívül néhány személyes adatot (név, születési év, lakcím, telefonszám) is eltárolnak. A lakcím egyetlen szöveges bejegyzés. Csak az aktuális kölcsönzéseket tartják nyilván a kölcsönzés dátumát is feljegyezve. Megjegyezzük továbbá, hogy egy olvasó egyszerre több könyvet is kölcsönözhet.

Tehát a tárolandó adatok:

- A könyvről eltároljuk
 - a könyv leltári számát,
 - a könyv címét,
 - a könyv ISBN számát,
 - a könyv kiadási évét,
 - a kiadót, aki kiadta a könyvet,
 - a könyv szerzőjét vagy szerzőit (feltételezzük, hogy több is lehet).
- Az olvasóról eltároljuk:
 - az olvasószámot (vagy másképpen az olvasójegyének számát), amelyet a könyvtárban kap,
 - a nevét,
 - a telefonszámát,
 - a születési dátumát,
 - a lakcímét.
- A kölcsönzéssel kapcsolatban pedig eltároljuk, hogy az adott olvasó mikor vette ki a könyvet.

Kezdjük az egyedek leképezésével!

Az Olvasó egyedből relációs adatbázissémát készítnék és felvesszük az attribútumait. A séma kulcisa az olvasójegy. Minden attribútum atomi, ezért különleges leképezési szabályt nem kell alkalmazni.

OLVASÓ(olvasószám, név, lakcím, születési dátum)

A Könyv egyedet is hasonlóképpen képezzük le, de figyelünk arra, hogy a Szerzők többértékű attribútum sémába vesszük fel. Mivel összetett attribútum is, ezért a részattribútumaival helyettesítjük. A Szerzők szerzőnév, a sorszám és a Könyv egyed kulcsa, vagyis a könyvszám kerül, ami a Szerzők sémájában a Szerzők táblában két összetett kulcs van, {könyvszám, sorszám} és {könyvszám, szerző}. Ezek a Szerzők attribútumpárt választjuk elsődleges kulcsnak.

Könyv(könyvszám, cím, kiadó, kiadási év)

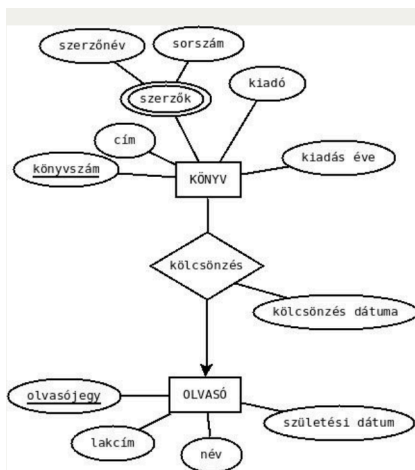
Szerzők(könyvszám, sorszám, szerző)

A következő lépésben a kapcsolatot képezzük le. Először ennek is felvesszünk egy relációs adatbázissémát a kapcsolódó egyedek kulcsait vesszük fel, valamint a kapcsolati attribútumot (kölcsönzés dátuma). A kölcsönzés leltári szám lesz, mivel 1:N kapcsolatról van szó, a könyv a kapcsolat N-oldalán van, és a kölcsönzés az Olvasó oldalán. Így a kölcsönzés a kölcsönzőtől a kölcsönzött könyv alapján meg tudjuk mondani, hogy kinél van.

Kölcsönzés(könyvszám, olvasószám, kölcsönzés dátuma)

Konszolidálás

Láthatjuk, hogy a Kölcsönzés és a Könyv séma kulcsa megegyezik, ezért ez a két séma össze



A könyvtári adatbázis E-K diagramja

A könyvtárban csak az aktuális kölcsönzéseket tartják számon. Ha egy könyv ki lett véve, akkor a sorában ki vannak töltve az olvasójegy és a kölcsönzés dátuma mezők. A szerzők mező egy sztringben tárolja a szerzőlistát.

Kölcsönzés({leltáriSzám, könyvCím, szerző, kiadó, kiadásiÉv, olvasójegy, kölcsönzésDátuma})
Olvasó({olvasójegy, név, lakcím})

{leltáriSzám} → {könyvCím, szerző, kiadó, kiadásiÉv, olvasójegy, kölcsönzésDátuma}
{könyvCím, szerző, kiadásiÉv} → {kiadó}
{olvasójegy} → {név, lakcím}

Kulcsok meghatározása

Tekintsük a {leltáriSzám}+ halmazt:

{leltáriSzám}* = {leltáriSzám, könyvCím, szerző, kiadó, kiadásiÉv, olvasójegy, kölcsönzésDátuma}
vagyis visszakaptuk a teljes attribútumhalmazt, így a leltáriSzám szuperkulcs és kulcs is.

A többi attribútum(halmaz) tranzitív lezártja, amelyben nem szerepel a leltáriSzám attribútum nem lesz kulcs, mert a leltáriSzám-ot nem tudjuk egyikből sem levezetni (nem áll egyik függés jobb oldalán sem).

1NF

Mindkét relációséma megfelel az 1NF-nek, mivel minden attribútumuk atomi (nincs bennük sem többértékű, sem pedig összetett attribútum).

2NF

A Kölcsönzés sémában egyetlen attribútum alkotja a kulcsot, így minden másodlagos attribútum teljesen függ a séma bármely kulcsától.

Az Olvasó sémában egyetlen attribútum alkotja a kulcsot, így minden másodlagos attribútum teljesen függ a séma bármely kulcsától.

Igy mindkét relációséma teljesíti a 2NF feltételét.

3NF

Az Olvasó sémában nincs tranzitív függés, így az 3NF-ben van.

A Kölcsönzés sémában megfigyelhető a {leltáriSzám} → {könyvCím, szerző, kiadásiÉv} → {kiadó} tranzitív függés, így a függés mentén bontjuk fel a sémát:

Kölcsönzés2({leltáriSzám, könyvCím, szerző, kiadásiÉv, olvasójegy, kölcsönzésDátuma})
Könyv({könyvCím, szerző, kiadásiÉv, kiadó})

BCNF

Az Olvasó relációséma BCNF-ben van, mert az {olvasószám} → {név, lakcím} függés nemtriviális és az olvasószám szuperkulcs.

A Kölcsönzés séma BCNF-ben van, mert a {leltáriSzám} → {könyvCím, szerző, kiadásiÉv, olvasójegy, kölcsönzésDátuma} függés nemtriviális és a leltáriSzám szuperkulcs.

A Könyv séma BCNF-ben van, mert a {könyvCím, szerző, kiadásiÉv} → {kiadó} függés nemtriviális és a {könyvCím, szerző, kiadásiÉv} attribútumhalmaz szuperkulcs.

Mutasd a végeredményt!

A normalizálás végén tehát három relációsémát kapunk:

Kölcsönzés2({leltáriSzám, könyvCím, szerző, kiadásiÉv, olvasójegy, kölcsönzésDátuma})
Könyv({könyvCím, szerző, kiadásiÉv, kiadó})
Olvasó({olvasójegy, név, lakcím})